



Desafio Técnico

Jornada de Pagamentos

Proposta de exercício referente à vaga em questão.

Referência	https://carreirasinternasitau.gupy.io/jobs/11941288
Formato	Case técnico individual, com entrega de solução simples e apresentação técnica
Foco	Backend, arquitetura distribuída, eventos, resiliência e observabilidade

Importante

Este documento contém apenas a proposta do desafio.

1. Contexto

A Jornada de Pagamentos opera em um ambiente distribuído, crítico e orientado a valor para o cliente. O desafio simula uma necessidade realista de evolução de uma plataforma de pagamentos, considerando modernização, estabilidade operacional, integração com canais digitais e responsabilidade sobre o que é entregue em produção.

Este desafio é referente à vaga indicada no link de referência deste documento.

2. Objetivo do desafio

Construir uma solução backend para registrar, consultar e acompanhar solicitações de pagamento, com separação clara de camadas, uso de eventos para integração assíncrona e tratamento adequado de falhas, rastreabilidade e estados de negócio.

A solução deve demonstrar domínio de engenharia backend em um contexto de arquitetura distribuída e modernização evolutiva, sem exigir uma implementação extensa ou excessivamente complexa.

3. Forma simplificada de execução

Para que o candidato consiga demonstrar conhecimento sem investir tempo excessivo, o desafio deve ser executado como um recorte funcional pequeno. O objetivo é evidenciar raciocínio técnico, organização de código, decisões arquiteturais e clareza de comunicação.

- Implementar apenas o fluxo principal de criação e consulta de solicitação de pagamento.
- Simular integrações externas quando necessário, deixando claro o contrato esperado.
- Usar banco local, containerizado, em memória ou alternativa simples, desde que a decisão esteja documentada.
- Representar a publicação de eventos por adapter local, mock, fila local ou mecanismo equivalente.
- Priorizar qualidade, clareza e testabilidade em vez de quantidade de endpoints ou funcionalidades extras.
- Documentar brevemente as decisões técnicas, limitações assumidas e possíveis evoluções.

4. Cenário proposto

A squad precisa modernizar parte de uma jornada de pagamentos. Hoje existe um fluxo legado que recebe uma solicitação de pagamento, persiste os dados principais e aciona integrações externas para validação e efetivação. O objetivo do exercício é criar uma versão moderna desse fluxo, preparada para evolução, operação e integração por eventos.

O domínio do desafio é composto por solicitações de pagamento feitas por um cliente, com valor, moeda, identificadores de origem e destino, status do processamento e histórico mínimo de eventos relevantes.

A squad precisa modernizar parte de uma jornada de pagamentos. Hoje existe um fluxo legado que recebe uma solicitação de pagamento, persiste os dados principais e aciona integrações externas para validação e efetivação. O objetivo do exercício é criar uma versão moderna desse fluxo, preparada para evolução, operação e integração por eventos.

O domínio do desafio é composto por solicitações de pagamento feitas por um cliente, com valor, moeda, identificadores de origem e destino, status do processamento e histórico mínimo de eventos relevantes.

Após a criação de uma solicitação de pagamento, a solução deverá disponibilizar um mecanismo de publicação de eventos para integração com outros sistemas. A implementação poderá utilizar uma tecnologia real de mensageria ou uma abstração equivalente, desde que demonstre claramente a separação entre domínio e infraestrutura.

5. Escopo funcional

- Criar uma solicitação de pagamento com dados mínimos de identificação, valor, origem, destino e contexto da operação.

- Consultar uma solicitação de pagamento por identificador único.
- Atualizar o status de uma solicitação conforme o avanço do processamento, quando esse fluxo for implementado.
- Publicar ou registrar evento de domínio quando uma solicitação for criada e quando houver mudança relevante de status.
- Disponibilizar documentação dos contratos da API para consumo por frontend ou outros sistemas.
- Registrar histórico mínimo dos eventos ou mudanças relevantes da solicitação.
- Disponibilizar contratos de API claros para consumo por outros sistemas ou por um frontend.

6. Requisitos técnicos

A implementação deve utilizar, preferencialmente, Java ou Kotlin. O candidato pode escolher frameworks e bibliotecas, desde que a escolha seja coerente com o problema e facilite evolução, testes e operação.

- Aplicar princípios de arquitetura hexagonal, separando domínio, portas, adaptadores e integrações externas.
- Modelar eventos de domínio relacionados à criação e atualização do pagamento.
- Prever integração assíncrona com mensageria, como Kafka, SNS/SQS ou mecanismo equivalente simulável localmente.
- Persistir dados de forma estruturada, utilizando PostgreSQL, DynamoDB, DocumentDB ou alternativa local equivalente para execução do exercício.
- Expor endpoints REST ou interface equivalente para criação e consulta de solicitações.
- Incluir testes automatizados relevantes para o domínio e para os componentes principais.
- Incluir logs estruturados, correlação de requisições ou pontos mínimos de observabilidade.
- Tratar erros de negócio e falhas técnicas de forma explícita e compreensível.

7. Requisitos não funcionais

Categoria	Expectativa
Resiliência	A solução deve considerar falhas em integrações, indisponibilidade temporária e tratamento equivalente.
Escalabilidade	A arquitetura proposta deve permitir crescimento do volume sem acoplamento excessivo.
Observabilidade	Logs, métricas ou rastreabilidade devem permitir entendimento do fluxo principal e investigação de incidentes.
Evolução	O desenho deve facilitar inclusão de novas regras, status e integrações sem reescrita ampla do núcleo de domínio.
Qualidade	Código legível, testável, organizado e com responsabilidade clara entre componentes.

8. Entregáveis esperados

- 1 Repositório ou pacote contendo a implementação do serviço backend.
- 2 Arquivo README com instruções objetivas para executar a aplicação, testes e dependências locais.
- 3 Declaração de uso de IA no README, informando ferramentas utilizadas, prompts principais e partes da solução geradas, revisadas ou apoiadas por IA.

- 4 Arquivo ADR.md com as principais decisões arquiteturais, trade-offs, limitações assumidas e possíveis evoluções.
- 5 Descrição breve da arquitetura adotada e das principais decisões técnicas.
- 6 Documentação dos endpoints disponibilizados, podendo ser apresentada através de OpenAPI/Swagger, coleção Postman ou mecanismo equivalente.
- 7 Evidências de testes automatizados e cenários cobertos.
- 8 Evidências mínimas de observabilidade, como exemplos de logs, correlação ou métricas propostas.

9. Restrições e premissas

- Não é necessário consumir serviços reais de nuvem pública para concluir o desafio.
- Integrações externas podem ser simuladas, desde que o desenho deixe claro o papel de cada integração.
- Mensageria pode ser executada localmente, em container, mock, fake ou mecanismo equivalente.
- O candidato deve priorizar clareza, consistência e qualidade da solução em relação à quantidade de funcionalidades extras.
- Dados sensíveis ou reais não devem ser utilizados no exercício.

10. Etapa 2 - Entrevista de apresentação técnica da solução

Na etapa 2, o candidato deverá realizar uma apresentação técnica da solução com duração de 20 minutos. Durante a apresentação, o candidato deverá demonstrar a solução desenvolvida, explicar as principais decisões arquiteturais e mostrar como o sistema se comporta nos fluxos principais. A conversa também poderá explorar trade-offs de modernização, operação em produção, tratamento de incidentes, evolução futura e colaboração com times multidisciplinares.

11. Tópicos que poderão ser discutidos

- Separação de responsabilidades entre domínio, aplicação e infraestrutura.
- Modelagem de estados de uma solicitação de pagamento.
- Estratégia para publicação e consumo de eventos.
- Tratamento de idempotência, duplicidade e reprocessamento.
- Escolha de persistência e impacto no modelo de consulta.
- Observabilidade para investigação de falhas em ambiente produtivo.
- Estratégia de testes e qualidade de código.
- Caminho de evolução da solução em uma arquitetura distribuída.